

TEMPLE TRAP

STORY OF THE LOST ONE

1.1 Abstract

Temple Trap is a sliding block puzzle inspired by an ancient temple adventure theme, where an explorer must escape a perilous maze of shifting stone pathways. The game challenges players to move interconnected tiles and stairways within a fixed grid to create a continuous path to the exit. Each move requires careful spatial reasoning and strategic planning, as one wrong slide can lead to a dead end or trap. It effectively combines logic, problem-solving, and thematic storytelling to create an engaging puzzle experience. This paper shows an implementation of the game, which is solved with the help of search algorithm and various piece-wise and movement constraints.

1.2 Introduction

Temple Trap is an intricately designed puzzle that stands out for its elegant and compact **state representation**, making it highly suitable for algorithmic modeling. To computationally solve this problem, I have employed the *A (A-star) search algorithm**, which efficiently explores possible configurations to find the optimal path from the starting position to the exit. In this formulation, **each tile piece**, along with all its possible **orientations and connections**, is treated as a **variable**, capturing the puzzle's combinatorial complexity. The **board configuration**, consisting of all tiles



placed in fixed orientations but allowed to move within the grid, defines a **state** in the search space. By systematically evaluating these states using the A* heuristic, the solver identifies the sequence of tile movements and player transitions that lead to the successful escape from the temple.

1.3 Variables

Variables have been implemented with the use of object-oriented programming in python. They store the following information:

- Side openings
- Level (Ground=0 and Flat=1)
- Pawn Indicator
- Blank Indicator
- Stair Indicator
- Tile name

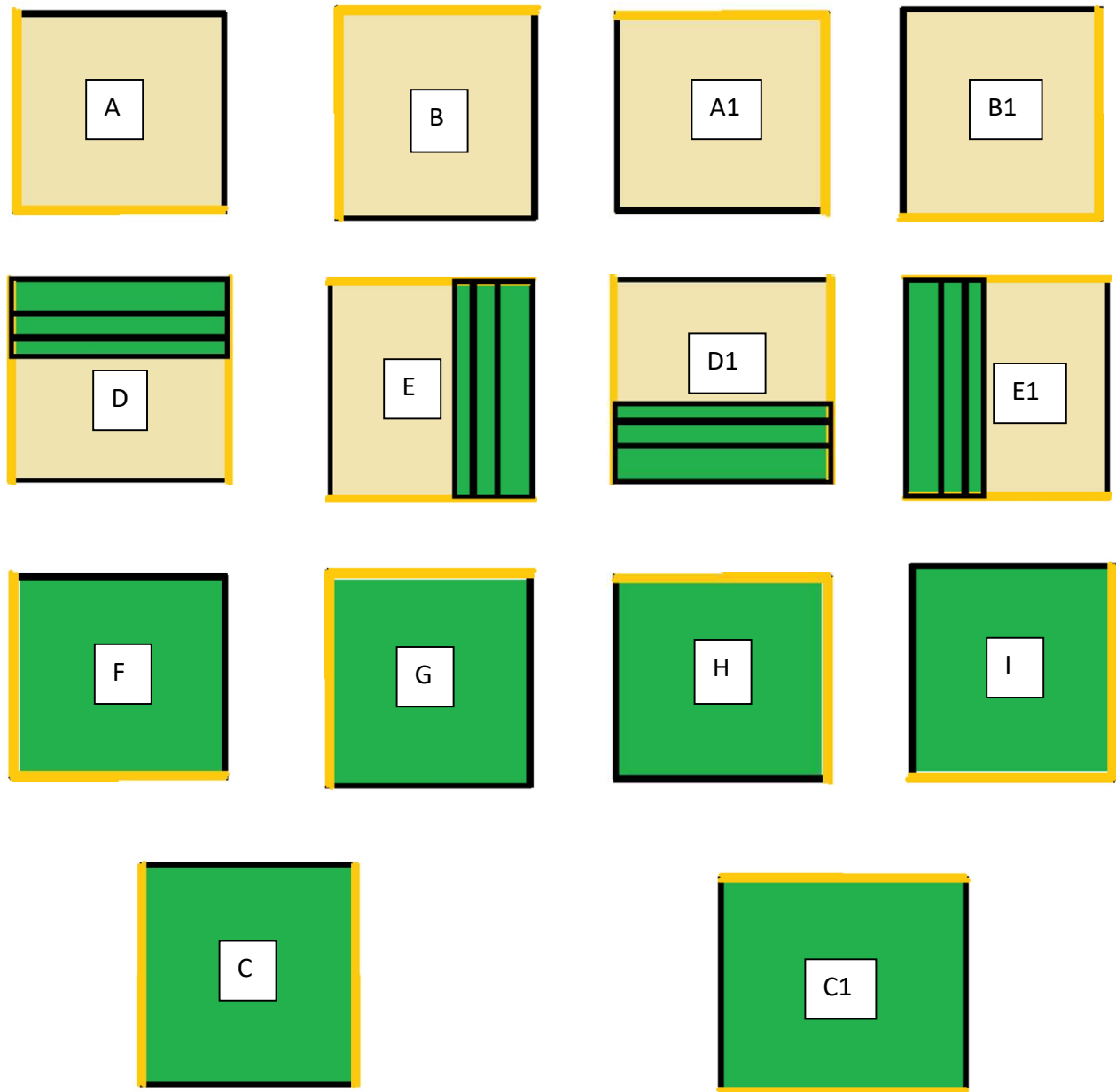


Fig 1.1 Showing different tiles with all different orientations.

1.4 State

States also have been represented with the help of classes in python. It is the varied arrangement of all pieces in 3*3 grid. It consists the information regarding:

- Arrangement of pieces

- Pawn Position
- Blank Position
- State Cost

1.5 Rules

1. Pawn's Move:
Before any slide, the Pawn may freely walk in the region reachable from its current cell. This includes movement on the Ground layer, climbing stairs, and traversing connected Top tiles. The Pawn may only rest on tiles that contain a hole.
2. Lock rule: A tile with a hole can slide into the blank only if the Pawn is not currently located on that tile.
3. Cost: Sliding one tiles has cost 1 and Pawn movement has also a cost of 1.

1.6 The Solver

This set of functions consists of three main functions and various other helper function to those three functions. The three main functions that include the crux of the problem are:

1. Pawn Movement: Creates a new state with single step valid pawn move.
2. Tile Movement: Creates a new state by swapping the tile and the blank.
3. Heuristics: Required for searching the Heuristics function

1.7 The A* search

This function includes the main A* search algorithm along with helper functions such as:

1. Goal Test
2. Path Identifier

1.8 Other Functions

1. Initialize Board: It uses the input to get the tile map and uses the variables to map the correct tile to its desired variable.
2. Visualize Path: It is a function that uses the "pygame" library to show the tile and pawn moving animation. The animation is quite minimal.

1.9 Input

Input should be a 3*3 matrix with the tile names which are predefined. In each row three variables must be entered which are separated by space, and the process should continue till the time nine entries are given that is three rows.

1.10 The Heuristics Function

Here we use simple Manhattan distance as heuristics ("-1") and some penalty for being on ground with some credit for being at the stairs.

1.11 State Space Expansion

For expanding the state-space, the movement in all the four directions is considered in both the pawn movement and the tile movement function. Tile are simple swapped with the blank whereas for the pawn movement all the movement are considered and only the allowed ones are carried out. New states are explored and then expanded.

1.12 Goal Test

When the Pawn position is at cell 0 and level 1 and either of the openings of the tile is at West. This is the test for goal state.

1.13 Board Initialization and Visualization

A function for board initialization is defined to link the entered texts to their respective variables.

A path visualizer helps to visualize the stored optimal path via a library called "Pygame."

1.14 Files

The program has been divided into set of two files namely "solver" and "visualizer" which are interdependent was running the complete program.

Solver consists the solver functions that generate state and the "A*" search algorithm.

Visualizer consists of functions for visualization of the complete path.

1.15 Gen-AI Disclosure

The use of generative AI has been made for defining some functions and a little debugging. Also, a slight help in optimization for levels above expert was taken.

1.16 Running Algorithm

Run in the terminal after installing "pygame". Simply save the two files together and run "solver.py" in terminal. It will take of "visualizer.py". A window of "pygame" would open to show the illustration.